

Counting Versus Testing the Destination: A Simulation Study of Layered Defences Against SMS OTP Flooding and Pumping

Naveed Ahmed^{a,*}

^a*Affiliation to be completed before submission, Saudi Arabia*

Abstract

An endpoint that sends a one-time password by SMS to any number it is given will be abused, to waste money (flooding) or to earn part of the termination fee (SMS pumping). Starting from an incident its operator recalls, we evaluate in simulation a twelve-step defence that derives trust from app attestation, signed sessions and verification outcomes, against simpler designs and 25 attacker profiles. Attacks with a signature in the request are contained at little cost to simulated users; a residential attacker with farmed CAPTCHA scores is rationed, not separated, by refusing half of first-time users. On the destination, short-window counters of SMS sends per number block, selected at a benign service target, leaked 6 to 48 messages across four concentrated 20-minute pumping profiles, against 19 to 452 for sequential tests of verification outcomes, at 0.06 to 0.43 points of attack-free completion loss. Under a second protocol, fixed before its runs, the selected counter met its leakage, benign-cost and relative-leakage claims on new seeds and shifted workloads, and failed both claims about service under attack when the pumper shares blocks with real users. It did not stop pumpers that spread over 30 or more blocks, paced themselves to its quota or built trust on random numbers, and it cost real users once their own traffic on a block approached the quota.

Keywords: SMS pumping, artificially inflated traffic, one-time password, rate limiting, risk scoring, residential proxies, sequential tests, fraud prevention

*Corresponding author.

Email address: optimistbs@gmail.com (Naveed Ahmed)

1. Introduction

Most online services verify a phone number by sending it a code by SMS, from an endpoint reachable before any account exists, at a tenth of a cent to twenty cents a message. A flooder makes the service send to numbers it does not own; a pumper sends to a range whose operator shares the termination fee, turning the service’s SMS budget into its income [1]. Twitter’s owner said in 2023 that pumping had cost the company about 60 million dollars a year [2], and Huh et al. found as many attack requests as genuine ones in three months of one large service’s logs [3].

This paper starts from such an incident on a registration flow the author operated, and from the redesign that followed: a twelve-step pipeline with attestation-backed platform exemptions, signed sessions, number checks before a message is paid for, key reputation built from whether codes were entered, a risk score with five responses, and a hard budget. With no production logs, we ask in simulation which attacks it contains against simpler designs, at what cost in completed registrations (RQ1); which layer is responsible (RQ2); when outcome-based reputation separates an attacker from simulated users (RQ3); what rationing costs when it does not (RQ4); and what destination policies cost legitimate users, with and without an attack (RQ5).

The destination became the centre of the study, because a pumper is paid only on its partner’s ranges: under a predeclared comparison, a plain counter of sends per number block leaked less than our sequential tests of what happened to the codes, and a second predeclared protocol mapped where that holds and where it fails. We contribute the incident as eight named gaps, the design and an open implementation, measurements of where reputation stops separating and what rationing depends on, a closed-form leakage estimate, both destination-policy comparisons with their failures, and a collateral-damage study.

2. Background and related work

Revenue-share fraud. In international revenue share fraud, traffic is generated to an advertised number range and the termination fee is split along the route [4]. Sahin and Francillon’s 3.14 million test numbers from twelve providers show the range to be the fraudster’s managed asset, about 80% of ranges being shared across providers [5].

36 *Detecting artificially inflated SMS.* Huh et al. trained gradient-boosted trees
 37 on 9.4 million SMS authentication logs from nineteen countries, reaching
 38 89.6% recall at 0.19% false positives on web traffic [3]. About 73% of web
 39 attacks sent four or fewer requests per identity a day, defeating per-entity
 40 limits, and destination signals were the strongest: about 92% zero conversion
 41 for attacks against 21% for genuine traffic. Their detector learns from labelled
 42 traffic and acts per request; our policies are hand-specified per destination
 43 block, and theirs was not reproduced.

44 *Defences in practice.* Twilio’s Fraud Guard blocks regions known for pump-
 45 ing [6]; its SMS Pumping Risk Score rates numbers from carrier, traffic and
 46 conversion signals and advises friction from 60 and refusal above 90 [7], so
 47 graded destination action is established practice; no product was bench-
 48 marked. A token bucket separates a sustained rate from a burst allowance;
 49 in RFC 2697 an excess bucket fills only when the committed one is full [8].
 50 CAPTCHA solving is a labour market at about a dollar per thousand [9],
 51 open where machine solvers fail [10]; client puzzles [11] were not evaluated.
 52 Residential proxy addresses rotate within minutes [12], and carrier-grade
 53 NAT makes address reputation on cellular networks “all but infeasible” [13].

54 *Identity and change detection.* One entity can present many identities [14];
 55 a signed session gives continuity, not scarcity. Our block tests are Wald’s se-
 56 quential tests [15] run as Page’s cumulative sums [16]. Quickest change detec-
 57 tion minimises delay subject to false-alarm constraints [17], and a CUSUM’s
 58 parameters are set from the error rates it must meet [18]; windowed statis-
 59 tics and confidence sequences [19] were not evaluated. Step 7 resembles
 60 risk-based authentication [20]; attacks on the code once sent are a separate
 61 problem [21, 22].

62 **3. The incident and the adapted attacker**

63 This section is motivation, not evidence: no logs from the period are
 64 in our hands, permission to describe the incident is being sought, and no
 65 result depends on it. The registration flow sent an SMS to any unregistered
 66 number. The first controls, v1, were reCAPTCHA v3 on web requests, one
 67 message per number per minute, five requests per address per minute, a
 68 country allow-list, a body-length limit, a proxy block and per-platform caps.
 69 The returning abuse used thousands of residential addresses, carried the iOS
 70 header to skip the CAPTCHA, and stayed under every limit.

Table 1: The v1 controls and their gaps, recalled by the operator from the incident (Recalled; not supported by logs) or found in the design review that followed (Review).

Gap	Source	v1 assumed	How it fails
A	Recalled	A platform header says which app sent the request	Sending the header reached every exemption
B	Recalled	Five requests per address per minute bound an attacker	A fresh residential address per request
C	Recalled	A number given is worth sending to	Generated numbers cost as much as real ones
D	Review	An allowed country is a safe destination	Premium ranges inside it cost far more
E	Recalled	Per-key caps bound total spend	A wide attack under every cap
F	Review	Check-then-increment is a rate limit	Two racing instances both pass
G	Review	Bulk and test flags are internal	Neither needs stronger authentication
H	Review	Each check says which check failed	Thresholds can be probed one at a time

71 *Threat model.* The attacker wants as many messages sent as possible, or
72 sent to numbers that pay it. It buys residential proxies (about 3 USD per
73 gigabyte) and human-like reCAPTCHA tokens (about 0.3 cents), mints fin-
74 gerprints, generates numbers and knows the thresholds. A pumper also has
75 a colluding operator’s ranges; it may enter the codes delivered there, report
76 deliveries as failed, build verified history, spread over many blocks and buy
77 challenge solutions. It cannot forge an attestation, use ranges its partner
78 does not terminate, or make a stranger enter a code.

79 4. Design

80 No message is paid for before every check passes, and cheap checks run
81 first. Trust comes from evidence the client cannot forge, an attestation, a
82 server-signed session and the fate of earlier codes, though such evidence can
83 be manufactured (Section 7.8).

84 4.1. The pipeline and the risk score

85 A request passes the steps of Table 2 until one refuses it. Step 0 takes
86 the platform only from a valid App Attest or Play Integrity result, ignor-
87 ing the header. Each address limit, per-number slot, budget reservation and

88 per-block counter slot is one atomic Redis operation, released if a later step
 89 refuses; the enclosing steps also make separate writes (Section 6). Every out-
 90 come after Step 0 except the challenge returns one HTTP 200 body padded
 91 to 400 ms.

Table 2: The twelve steps and the v1 gap each targets. FB is the feedback loop.

Step	Check	Gap
0	Proxy block; platform from attestation, never from the header	A
1	Signed session bound to a fingerprint; 3 requests per session per 10 min	B
2	Limits per address, /24 and ASN, lower for hosting ASNs	B, F
3–4	reCAPTCHA v3 threshold and origin check, web only	–
5	Number intelligence: cost class, sequential numbers, liveness; reads the block’s verdict stage	C, D, G
6	Body length, excluded numbers; test bypass needs a credential	G
7	Risk score and tier: allow, delay, challenge, downgrade, block	H
8	Per-number backoff, 60 s doubling to 1 h, five a day	–
9	Per-source, platform and country caps with an adaptive multiplier	–
10	Operating mode: elevated above 80% of the hourly budget, emergency at 100%	E
11	Reserve hourly count, spend and any destination slot; write state; enqueue; uniform response	E, H
FB	Verified, failed, undelivered per key; destination-block tests; denylist of fine keys	–

92 Step 7 computes

$$s = \sum_i w_i x_i + w_c \frac{\max(0, \theta - r_{\min})}{\theta} + w_s f + w_m i - w_v v_n - w_f v_f, \quad (1)$$

93 where the x_i are request signals (CAPTCHA score, hosting ASN, finger-
 94 print age, session requests, country mismatch, number class), $r_{\min}$ is the
 95 lowest conversion ratio over the request’s keys (address, subnet, ASN, fin-
 96 gerprint, country, prefix, destination block), $\theta = 0.3$, $w_c = 25$, f and i flag
 97 a flooded key and one whose codes are entered within seconds, and v_n , v_f
 98 mark a number and fingerprint with verified history; other weights are 5 to 25
 99 points [23]. Tiers start at 20, 40, 60 and 80: *allow*; *delay* (a doubling wait);
 100 *challenge* (credits -20 when solved); *downgrade* (push or WhatsApp, never
 101 SMS); *block*. A datacenter address with a fresh fingerprint and a middling
 102 CAPTCHA score lands in *challenge*.

103 4.2. The feedback loop

104 A send resolves as *verified* when its code is entered, *failed* after five wrong
 105 attempts or two minutes past the delivery receipt, and *undelivered* without
 106 a receipt in sixty seconds or with a failure receipt; only the first two feed
 107 reputation. A positive receipt after a negative one reopens the send while its
 108 code is valid; a late verification reclassifies it, but an issued verdict stands
 109 until it expires. Each transition records its effects in the same compare-and-
 110 set and a recovery sweep finishes a dead process’s work, so effects apply at
 111 most once, and exactly once if the sweep runs within 20 minutes, with one
 112 exception (Section 6).

113 4.3. Destination policies

114 A pumper is confined to the ranges its partner terminates but can rotate
 115 numbers and blocks within them. Destination policies are keyed on the 8-
 116 digit block (10 000 numbers), so their protection weakens as a pumper spreads
 117 over more blocks.

118 *Sequential tests.* The conversion statistic assumes users verify with $p_\ell = 0.8$
 119 and a flooder with $p_a = 0.1$: a failure adds $\ln \frac{1-p_a}{1-p_\ell} = 1.50$, a verification
 120 $\ln \frac{p_a}{p_\ell} = -2.08$. The speed statistic assumes a machine enters a verified code
 121 within five seconds with $q_a = 0.9$ and a person with $q_\ell = 0.2$, since autofill
 122 removes the keystroke [24, 25]; it adds 1.50 or -2.08 . A verdict is issued,
 123 and the statistic reset, when either exceeds $h = \ln 1000 = 6.91$. The floor
 124 $S_t = \max(-ch, S_{t-1} + \text{increment})$ sets the memory: unbounded c is Wald’s
 125 test, $c = 0$ Page’s cumulative sum, and any finite c a zero-floor CUSUM
 126 with threshold $(1 + c)h$ and head start ch ; the default is $c = 1$. A first
 127 verdict makes the block’s first-time clients pass a challenge for an hour; a
 128 second within the hour moves them off SMS, refusing those without another
 129 channel. The stage is decided in the crossing’s atomic write.

130 *Counters.* A counter limits SMS sends per block in a daily or short refilling
 131 window. Step 11 reserves a slot, and decides the stage, in one atomic oper-
 132 ation when an SMS is about to go out, so a refused or challenged request
 133 costs nothing and concurrent requests cannot skip the challenge. Past the
 134 limit a graded counter challenges first-time clients, and past twice the limit
 135 moves them off SMS; a refusing counter refuses. Clients with verified history
 136 are exempt from both policies, so a code entered on a pumper’s number can
 137 buy that number an exemption. A token bucket of the same rate (burst 4 or

138 12, with a challenge-tier bucket that refills independently, unlike RFC 2697’s
 139 excess bucket) and the counter combined with the sequential tests were run
 140 as alternatives.

141 4.4. Capacity controls

142 Step 9 multiplies its caps by 0.25 to 1.5 according to a baseline job com-
 143 paring each key with its expected rate, learned from the pipeline’s hourly
 144 counters as a weekly profile or from the closed hours available; with none,
 145 the static cap stands. Verified history avoids the adaptive reduction, not the
 146 static caps or budget, and the budget bounds SMS count and spend units,
 147 not vendor or fallback costs.

148 5. An estimate of pumping leakage

149 Against the sequential tests, a pumper reaching B blocks at rate λ with
 150 N requests leaks about

$$L \approx \min(N, kB + \lambda\tau), \quad k = \lceil h/1.50 \rceil = 5, \quad (2)$$

151 where τ , the time a send stays unresolved, is about 2.5 minutes if the carrier
 152 never verifies and seconds if it verifies at once. It assumes no pre-existing
 153 credit, no solved challenges, no verified-history exemptions and no outage
 154 suspension, and predicts the first verdict, not containment under graded
 155 action. Evaluated on each run’s observed blocks it checks the mechanism;
 156 once $kB > N$ it only says everything leaks. A carrier verifying a share σ
 157 of codes with human-like delay gives the conversion statistic an expected
 158 increment of $\sigma(-2.08) + (1 - \sigma)(1.50)$, zero at $\sigma^* = 0.42$.

159 6. Implementation and evaluation method

160 *Implementation.* About 3 700 lines of Python implement the pipeline, feed-
 161 back loop and vendor adapters (App Attest enrolment tested only on syn-
 162 thetic certificate chains) on in-memory and Redis backends. Of 393 tests,
 163 twelve run two instances on a real `redis-server`, covering the hourly ceiling,
 164 number claims, callback counting, graded escalation, crash recovery, mixed
 165 receipts and the counter’s boundaries under concurrency; eight concurrent
 166 requests at a four-send limit give four SMS and four challenges. These test
 167 specific cases, not all interleavings. Known open defects, none exercised by

168 the single-threaded simulator: Step 11’s writes after its reservations are sep-
 169 arate, so a crash leaks reserved slots or resolves the send as undelivered; a
 170 correcting receipt landing between the timeout worker’s decision and its re-
 171 moval of the timeout leaves the send unresolved; a late verification recovered
 172 more than about thirteen minutes after it arrives can leave the failure it re-
 173 verses counted; and a request challenged at Step 11 keeps the source-cap slot
 174 it used.

175 *Simulation.* The pipeline runs against scripted vendor fakes on a simulated
 176 clock. After a warm-up of ten minutes (30 or 130 for pumping), each run has
 177 twenty minutes of attack (60 or 360 where stated) over legitimate traffic at 20
 178 requests per minute. Simulated users verify 80% of codes after a lognormal
 179 delay (median 22 s; 2 s for the 20% using autofill); 70% of numbers are reach-
 180 able on WhatsApp, a stable property of the number; 90% of users complete
 181 a challenge; 20% are returning account holders with a stable fingerprint, a
 182 trusted number and verified history. Legitimate and attacker requests use
 183 separate random streams, so at a seed every scenario offers the same users,
 184 as tests assert.

185 Behavioural parameters are assumptions, annotated with any published
 186 figure behind them, such as one provider’s global conversion rate of 68% or
 187 more [26]. The main study uses thirty seeds per attacker (Table 3) and draws
 188 pool size, rate and CAPTCHA class per seed; in a nested design (ten configu-
 189 rations, three seeds each) these draws account for 92% [80, 98] of the variance
 190 of the farm’s leakage. Other studies use ten seeds unless stated. Brackets
 191 are 95% percentile-bootstrap intervals, paired per seed for differences; they
 192 describe variation under this generator, not uncertainty about real traffic.

193 *Two protocols and their chronology.* The first protocol, committed before
 194 its runs, fixed a *matched comparison* and a *robustness study*. The matched
 195 comparison changes only the destination policy and, at three calibration
 196 settings of legitimate block density, selects on three tuning seeds the set-
 197 ting with the least leakage summed over four concentrated pumpers among
 198 those within 0.5 points of the default’s completion; the full sequential grid
 199 was searched only at the busiest setting. The robustness study fixes twelve
 200 Latin-hypercube points over conversion (0.6–0.9), autofill, CAPTCHA scores,
 201 WhatsApp reachability (0.3–1.0), returning share and legitimate rate, and
 202 a held-out family of attack rates (60 to 120 per minute), pools (50 000 to
 203 500 000 addresses) and heterogeneous legitimate traffic, with claims C1–C5

204 that hold if true in 90% of cells. Its first run failed C1 in 32 of 108 cells
205 because attack and no-attack runs drew different users; we fixed the pairing,
206 a fix that followed a result, and reran every study.

207 The second protocol was written after the matched comparison had se-
208 lected three counters. It froze them and their comparators and tested them
209 on new seeds (300–309), on the held-out family (seeds 6000 and up, six of
210 which had served other robustness points, never selection), on attacked traffic
211 shared with real users, against the adaptive attackers, and across an oper-
212 ating boundary of block load and pumper spread, with claims K1–K5 under
213 the same pass rule; it was committed before its driver and runs. The main
214 evaluation (9 919 runs) and the counter study (7 925) are clean invocations
215 whose runs carry the code hash.

216 *Designs and metrics.* Feature flags give seven designs: **v1** (header-trusted
217 platform, static caps), **v1-corrected** (platform from attestation), **budget-**
218 **only** (sessions, static caps, budget), **block-limit** (sessions, a refusing counter
219 of five sends per block per day), **conversion-only** and **speed-only** (v2
220 minus one block test), and **v2**. A seed is *contained* when leakage stays at
221 most 5% of the attack rate from some minute to the window’s end, for at
222 least five minutes. Requests that met a verdict and never completed are
223 a *descriptive* count; *attributable* loss compares the same requests with the
224 policy enforced and absent, net of the reverse, in points of offered users, on
225 attack-free traffic (benign) or on the attacked trace.

Table 3: The 25 attacker profiles by capability, and the destination policies each faced: S, sequential tests; C, counters; B, block-limit. Parameters: `results/attacker_profiles.md`.

Profile	Description	Policies
<i>A signature in the request</i>		
<code>naive_single_client,</code>	One address and session; hosting ASN abroad	S, B
<code>datacenter_rotation</code>	A premium-rate prefix; numbers walked upward	S, B
<code>premium_pumping, sequential_numbers</code>	The farm with the iOS header, unattested	S, B
<code>spoofed_platform</code>		
<i>Looks like a person</i>		
<code>residential_bot, _captcha_farm,</code>	Bot or farmed CAPTCHA; fresh or aged fingerprints	S, B
<code>_aged_fps</code>	One 48-hour-old browser profile	S, B
<code>residential_reused_profile</code>	2 or 10 requests/min over 20 ASNs; data-center rotation buying challenge solutions	S
<code>low_and_slow_{20_asns,</code>		
<code>below_dilution}, challenge_solver</code>		
<i>Games outcomes or trust</i>		
<code>pumper_verifies_{instantly,</code>	Elevated-cost or standard ranges; verifies within 1 s, or 60% after 30 s	S
<code>humanlike},</code>		
<code>pumper_standard_range_humanlike</code>		
<code>threshold_aware_carrier,</code>	Enters a code only when its block nears a verdict; reports every delivery failed	S, C
<code>receipt_faking_carrier</code>	500 identity/number pairs verify for 10 (or 30) min, then flood	S, C
<code>trust_building_{pumper,</code>		
<code>concentrated,long}</code>		
<i>Concentrated or spread on the destination</i>		
<code>concentrated_pumper_*</code> (four carriers)	Three blocks; carrier never verifies, verifies within 1 s or 60% after 30 s, or buys challenges	S, C
<code>block_poisoner</code>	Floods the blocks simulated users concentrate on	S, C
<code>spread and quota-aware pumbers</code> (generated)	3 to 300 blocks; at most the counter's quota per block	S, C

7. Results

7.1. RQ1: what is contained, and at what cost

Datacenter rotation leaks nothing under v2 because it scores in the challenge tier and declines challenges; Step 5 refuses the premium range; the session cap holds the single client to two messages (Table 4, Figure 1). In all 108 robustness cells these attacks leaked under 5% with completion within two points of the paired no-attack run (claim C1). The walk leaks 61 [53, 70] messages before its block's verdict at minute 3.0; the reused profile leaks 88 [83, 95], costing 4.9 points as it tightens the adaptive cap.

Block-limit stops the walk at five messages, better than v2's 61, and premium pumping at five, where v2 lets none through. The forged header buys nothing under v2, which ignores it; v1 puts the attacker in the app bucket, whose static cap holds it to 100 messages, so attestation removes the

Table 4: Leaked share of about 600 attack requests in twenty minutes (mean of per-run shares), thirty seeds: caps lifted (v1, block-limit BL, v2) and on (v1, v2, with first-time users refused and all users completing). Aged fingerprints behave as the farm. v1-corrected equals v1 except on the forged header (99.5; 96.0, 4.5, 76.4); budget-only equals v1-corrected except on the single client (0.3) and premium pumping (94.9; 94.5, 6.0, 75.3); conversion-only equals v2; speed-only equals v2 except on the walk (13.3; 12.3, 3.8, 77.3).

Attacker	lifted: leaked %			caps on, v1 (%)			caps on, v2 (%)		
	v1	BL	v2	leaked	refused	compl.	leaked	refused	compl.
Single client	5.6	0.3	0.3	5.6	0.9	79.3	0.3	1.1	79.1
Datacenter rotation	69.0	69.0	0.0	66.9	3.2	77.5	0.0	1.1	79.1
Premium pumping	99.4	1.0	0.0	95.7	4.9	76.2	0.0	1.1	79.1
Sequential numbers	99.4	1.0	11.5	95.7	4.9	76.2	10.7	3.5	77.5
Reused profile	99.5	99.5	22.5	96.0	4.5	76.4	17.4	8.6	74.2
Residential bot	37.2	37.2	37.2	37.2	0.9	79.2	29.9	16.2	69.6
Farmed CAPTCHA	99.5	99.5	99.5	96.0	4.5	76.4	45.6	54.0	45.5
Forged header	100.0	99.5	99.5	21.1	0.9	79.3	45.6	54.0	45.5

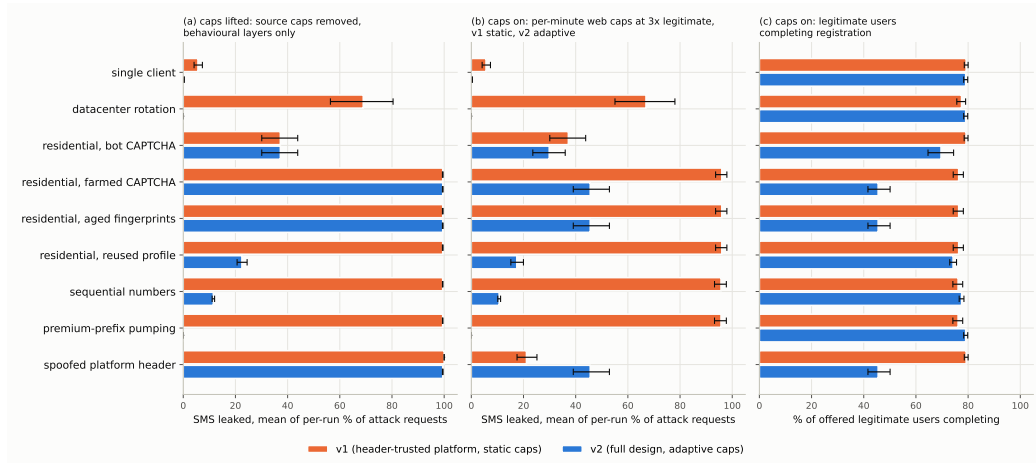


Figure 1: Leaked share of attack requests under v1 and v2: (a) source caps lifted, behavioural layers only; (b) per-minute source caps at three times the legitimate rate; (c) legitimate completion with caps on. Thirty seeds, 95% bootstrap intervals.

239 choice of bucket, not leakage. The farmed-CAPTCHA attacker leaks 99.5%
240 with caps lifted under every design and 96% under v1 with caps on; v2's
241 adaptive cap holds it to 229 [215, 244] messages, 46%, by refusing 54% [47,
242 60] of first-time users, so 45.5% complete instead of 76.4%. That is rationing;
243 claim C5 (v2 leaks at most 0.7 times v1 and refuses ten points more first-time
244 users) held in 36 of 36 cells.

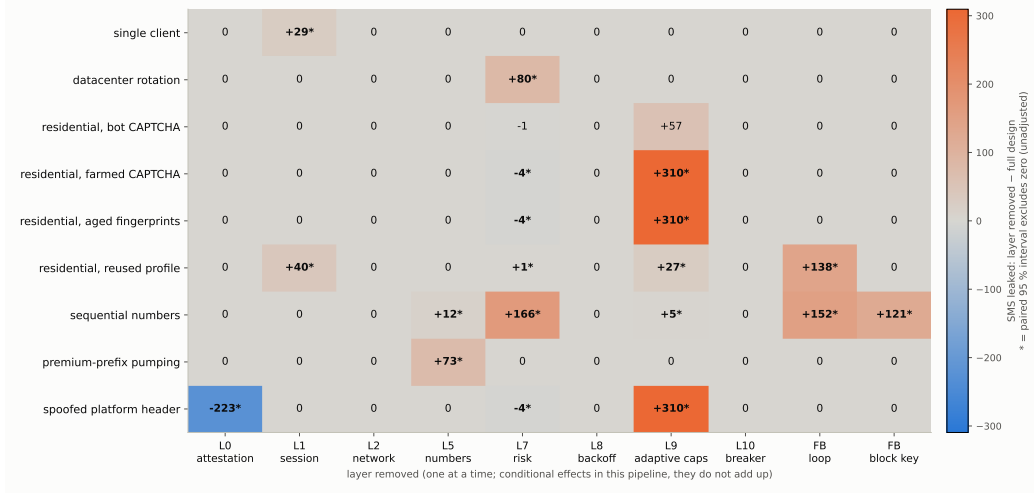


Figure 2: Ablation, caps on: leaked messages with one layer removed minus the full design, paired on ten seeds and one offered trace; * marks an unadjusted bootstrap interval excluding zero (19 of 90 cells). L0–L10 are steps; FB is the feedback loop and block key; all cells with intervals are in the artifact.

7.2. RQ2: conditional effects of each layer

Each cell of Figure 2 is a conditional effect, given all other layers, and overlapping effects do not add up. Removing the feedback loop and the block key together from the walk costs 152 messages, as removing the loop alone does, because the key acts only through the loop (interaction $-121 [-141, -101]$); the risk engine and the loop overlap likewise (-152), as do the loop and the session against the reused profile (-40), while the farm’s adaptive-cap effect does not depend on the risk engine ($+4 [-0, +10]$). Attestation’s negative effect (Table 5) is the bucket effect above.

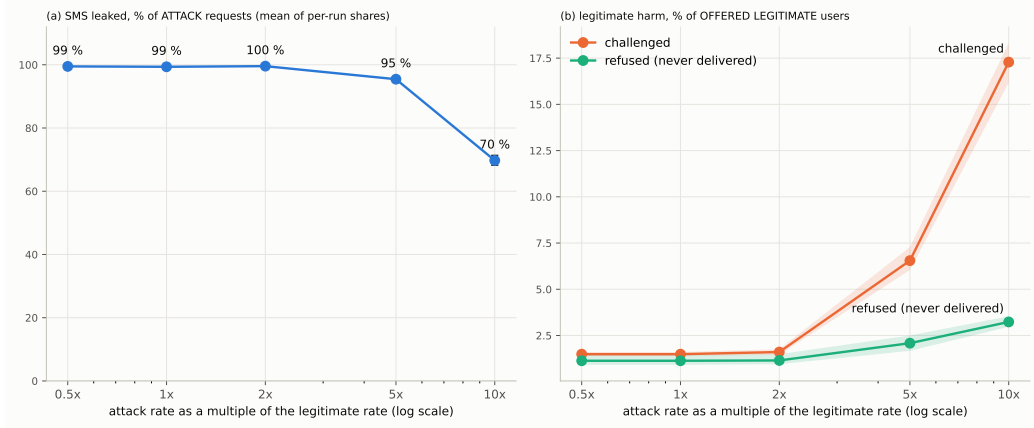


Figure 3: The farm at multiples of the legitimate rate for an hour, caps lifted, five seeds: (a) leaked share per run, intervals narrower than the markers except at 10 $\times$; (b) legitimate users challenged and refused.

Table 5: Conditional effects of at least ten messages whose paired interval excludes zero (caps on, ten seeds): extra messages leaked and change in completion when the layer is removed.

Attacker	Layer removed	Leaked (msgs)	Completion (pp)
Farm; aged fingerprints; forged header	adaptive caps	+310 [194, 421]	+28.7
Forged header	attestation	-223 [-250, -199]	+31.6
Sequential numbers	risk engine	+166 [154, 176]	-30.8
	feedback loop	+152 [144, 158]	-22.0
	block key	+121 [101, 141]	-7.7
	number intelligence	+12 [10, 14]	-0.8
Reused profile	feedback loop	+138 [122, 154]	-26.6
	session	+40 [30, 48]	-7.2
	adaptive caps	+27 [15, 39]	+4.6
Datacenter rotation	risk engine	+80 [69, 91]	-3.8
Premium pumping	number intelligence	+73 [56, 90]	-2.4
Single client	session	+29 [17, 41]	0.0

254 7.3. RQ3: where reputation stops separating

255 A farmed-CAPTCHA attacker uses an address and a fingerprint once and
256 shares its country, prefix and residential ASN with every legitimate user, so
257 the penalty its failures earn lands on everyone.

258 Up to twice the legitimate rate, 99 to 100% of the attack leaks (Figure 3):
259 a shared key stays above 0.3 conversion until failures outnumber legitimate
260 sends about $(0.8 - 0.3)/0.3 \approx 1.7$ to one, and the conversion penalty alone
261 reaches only *delay*. At ten times, 70% leaks while 17% of legitimate users are

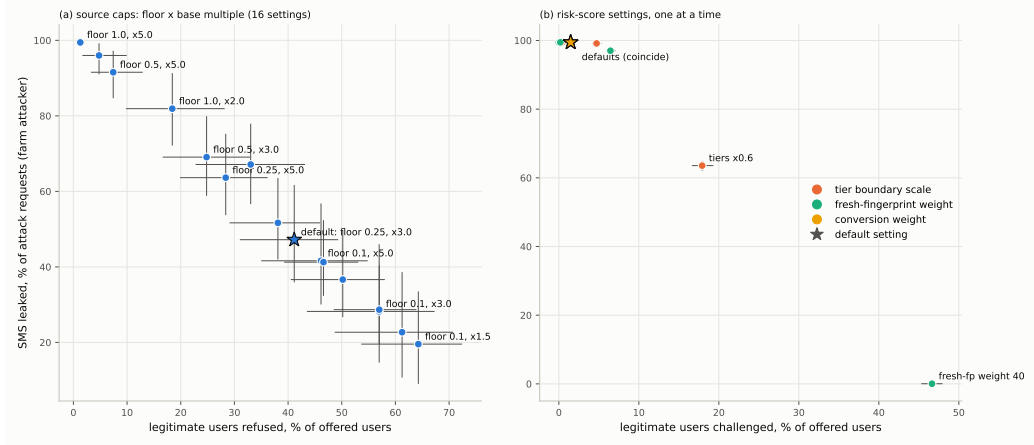


Figure 4: Farm leakage against legitimate users refused (a) and challenged (b), ten seeds: (a) cap floor and base multiple, caps on; (b) risk-score settings changed one at a time, caps lifted. Stars: defaults. Sampled points, not an optimised frontier.

262 challenged and 3.2% refused. Claim C2 (the farm leaks at least 90% with
 263 caps lifted) held in 34 of 36 cells. Changing the risk score one setting at
 264 a time (Figure 4b) separates no better: the conversion weight and window
 265 change nothing, and a fresh-fingerprint weight of 40 stops the attacker only by
 266 challenging 47% of legitimate users and refusing 8.5%, because most sign-ups
 267 are new browsers too. In the settings run, reputation separated an attacker
 268 only on keys its sends dominated [14].

269 7.4. RQ4: what rationing costs, and what it depends on

270 When separation fails, the cap is the lever (Figure 4a): the default (floor
 271 0.25, base three times the legitimate rate) lets 47% of the farm's requests
 272 through and refuses 41% of all legitimate users; the tightest setting, 20%
 273 and 64%. The gain depends on cadence and history (Table 6): a two-minute
 274 job rations less strongly, five- or ten-minute jobs depend on the tick's phase,
 275 and an hourly job leaks 91 to 96% even of a sixty-minute attack. The learned
 276 job leaves the static cap with no closed hour; a weekly profile poisoned at
 277 the attack's hour leaks 45 more messages, a stale one up to 106 more.

Table 6: Farm, caps on, ten seeds: (a) baseline-job cadence and tick phase, job told the legitimate rate; (b) the deployed job learning from three weeks of the pipeline’s counters, paired against the clean weekly profile.

(a) Cadence, 20-minute attack unless stated				
Cadence	Phase	Leaked	Completed	First-time ref.
1 min (default)	aligned	47.2%	47.5%	51.5%
2 min	aligned / offset	57.2 / 57.7%	53.6 / 54.3%	41.7 / 41.0%
5 min	aligned / offset	74.6 / 96.0%	63.8 / 76.2%	25.4 / 4.9%
10 min	aligned / offset	92.0 / 86.0%	75.1 / 71.5%	7.6 / 13.8%
hourly	either	96.0%	76.2%	4.9%
hourly, 60-min attack	aligned / offset	91.1 / 96.2%	73.5 / 76.4%	9.4 / 4.8%

(b) Learned baseline			
History	Leaked (msgs)	Paired diff.	Completed
weekly profile, 3 weeks	193 [161, 229]	–	40.5%
cold start (no closed hour)	533	+340 [+239, +441]	76.2%
three closed hours	235	+42 [+28, +60]	48.4%
stale, legitimate rate halved	299	+106 [+62, +151]	58.2%
stale, legitimate rate doubled	162	–30 [–44, –19]	35.7%
poisoned at that hour each week	238	+45 [+10, +77]	49.3%
two workers; restart at minute 10	193	+0 (identical runs)	40.5%

278 7.5. Pumping against the sequential tests

Table 7: Concentrated pumpers on three blocks, caps lifted, ten seeds: messages leaked of about 574 requests in twenty minutes (one 60-minute row); seeds contained in the window; mean minutes to containment (contained seeds) and to the first verdict (seeds with one); codes entered. Legitimate completion 79.1 to 79.2%; dash: not applicable or not reported.

Carrier, design	Leaked (msgs)	Contained (seeds)	Containment (min)	First verdict (min)	Codes entered
never verifies: v1	570 [419, 736]	0/10	–	–	0
v2 default	90 [71, 114]	10/10	3.5 [3.1, 3.9]	3.0	0
v2, 24-h denylist	90 [71, 113]	10/10	3.5 [3.1, 3.9]	3.0	0
verifies within 1 s: v2 default	19 [17, 21]	10/10	1.4 [1.1, 1.7]	0.4	19
verifies 60% after 30 s: v1	570	0/10	–	–	344
v2 default	452 [365, 539]	0/10	–	11.6	272
v2 default, 60 min	1067 [839, 1310]	2/10	–	–	–
buys challenges: v2 default	105 [86, 127]	10/10	4.3 [3.8, 4.9]	3.0	0
v2, 24-h denylist	90 [71, 113]	10/10	3.5 [3.1, 3.9]	3.0	0
farm (control): v2 default	567 [410, 745]	0/10	–	–	0

279 A pumper whose carrier never verifies is contained in all ten windows
280 at minute 3.5 after 90 messages (Table 7); one verifying within a second is
281 caught by the speed test at 19, and a challenge buyer leaks 105. The carrier
282 verifying 60% with human-like delay draws its first verdict at minute 11.6
283 on average; no window ends contained, 452 messages leak and 272 codes

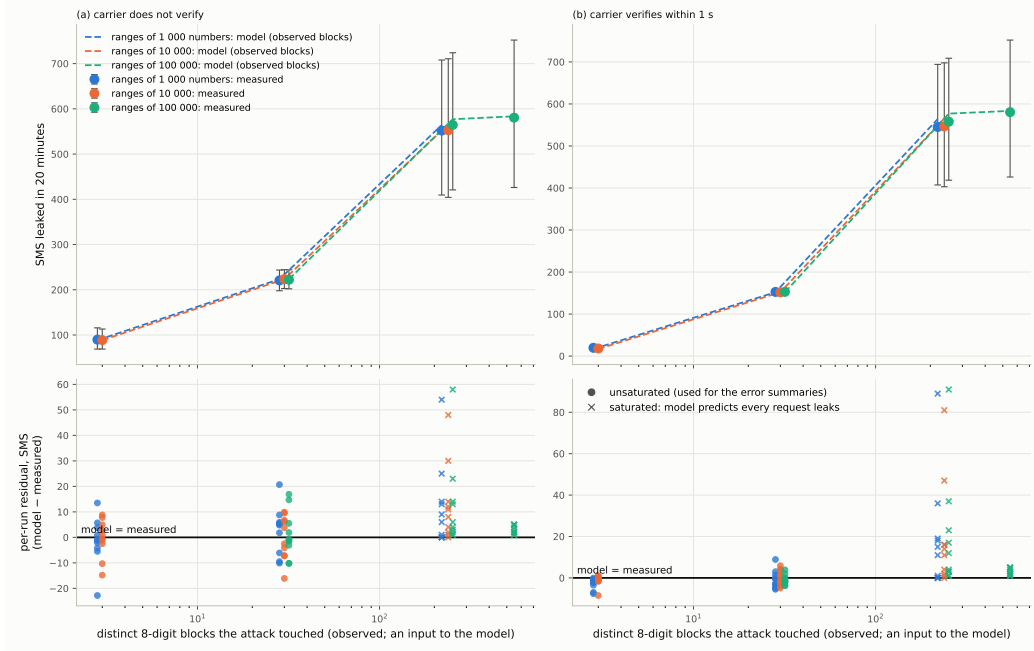


Figure 5: Pumper spread against the sequential tests, ten seeds: leakage against the blocks a run touched (observed, an input to Equation (2), dashed) for ranges inside, aligned with or spanning the 8-digit key; lower panels: every run’s residual (model minus measured, messages), crosses where the model predicts that every request leaks. A mechanism check on observed blocks, not a forecast.

are entered, and over sixty minutes 1067 leak with eight runs in ten never contained. Claim C3 (a never-verifying pumper is contained with at most 30% leaked) held in 36 of 36 cells.

Across 18 spread configurations, leakage follows the blocks touched (Figure 5): with 30 blocks five to seven runs in ten are still contained, with about 240 none. In the ten unsaturated configurations Equation (2) errs by 1.6 to 7.9 messages per run on average (1 to 13%); where saturated it overstates by up to 91.

292 *7.6. Destination policies: where the counter wins, and where it fails*

Table 8: Matched comparison at the busiest setting (200 blocks, 144 legitimate sends per block per day), selected on three tuning seeds, reported on ten others: twenty-minute leakage by never-verifying (NV), instant (IV), human-like (HL) and challenge-buying (CB) carriers; from 24-hour attack-free runs at 65% conversion, completion, net loss attributable to the policy (points), challenge rate and verdict events per day. Benign target: 63.95%.

Setting (selected as)	NV	IV	HL	CB	Compl.	Net loss	Chall.	Events
none	319	566	570	405	64.13%	–	1.53%	0
sequential <i>T</i> 1000, <i>c</i> 1 (default)	90	19	452	105	64.13%	0.01	1.62%	5.8
sequential <i>T</i> 300, <i>c</i> 0 (best at target)	88	14	126	98	63.79%	0.34	6.24%	314.3
counter, graded, 4 per 10 min (best)	24	24	24	48	64.07%	0.06	2.38%	0
counter, graded, 160 per day (best daily)	319	409	408	405	64.11%	0.02	1.87%	0

293 *At a benign service target.* Under the first protocol (Table 8, Figure 6), the
294 sequential setting with least leakage at the target holds the human-like carrier
295 to 126 messages only by raising 314 false verdict events a day; at the default’s
296 false-alarm budget every credit value leaks 452 or more. The graded counter
297 of four sends per block per ten minutes leaks 24 against each outcome-reading
298 carrier, more than the default’s 19 against the instant verifier, and 48 against
299 the challenge buyer, at 0.06 points (0.07 at 80% conversion). Counters se-
300 lected at the two sparser settings leak 6 to 18 at 0.15 to 0.35 points (up to 0.43
301 at 80%). Rerun on fresh seeds with the full sequential grid at every density,
302 the rule selects the same counters under mean or worst-case leakage, a first-
303 time-loss constraint, any single attacker or random mix; the middle setting’s
304 counter misses the target by 0.12 points, though no seed-and-conversion cell
305 falls 0.5 points below the default.

306 *Under a second protocol.* Frozen and run on the held-out family (Table 9),
307 the counter leaked 33, 34, 33 and 62 messages on average against the four
308 carriers, where the default leaked 287, 36, 344 and 287 and no policy 470
309 to 1857, at a benign cost of 0.13 [0.09, 0.17] points. It failed both service-
310 under-attack claims. Thirteen of the 17 cells that broke K3 lie at the three
311 points with the most legitimate traffic (34 to 39 requests a minute), where
312 users and pumper compete for one quota; K5 failed in 51 cells over 11 of 12
313 points. The counters selected for sparser traffic cost 4.7 and 5.8 points here
314 even without an attack: the quota must match the traffic.

315 On 60-minute attacks against traffic shared with real users at 200 blocks,
316 measured on the same attacked trace, the counter leaked 80 to 158 messages



Figure 6: The matched comparison. (a) Every setting evaluated at 200 blocks: attack-free completion against leakage summed over the four pumpers, with the benign target. (b) Per-attacker paired leakage differences from the default for the selected settings.

Table 9: The second protocol’s claims for the principal counter (graded, 4 per 10 min) on the held-out family: 12 shifted points, held-out attack rates and pools, three new seeds each, a 20-minute pumper on three blocks that real users also use. A claim holds if true in 90% of cells.

Claim	Statement	True / cells	Result
K1	Leaks at most 10% of attack requests	144 / 144	holds
K2	Attack-free attributable loss at most 0.5 points	36 / 36	holds
K3	Attributable loss under attack at most 0.5 points	127 / 144	fails
K4	Leaks no more than the default sequential tests	131 / 144	holds; all 13 misses are the instant verifier
K5	Attributable loss under attack at most the default’s plus 0.25 points	93 / 144	fails

317 against the four carriers, the default 31 to 897 and no policy 587 to 1981;
318 attributable loss ranged from -0.16 to $+0.09$ points of all users under the
319 counter and -0.21 to $+0.04$ under the default (negative: a gain). With no
320 destination policy the pumper exhausts the shared blocks and the hourly
321 budget, so the counter gained the attacked blocks’ users 11 points against
322 the never-verifying carrier. A poisoner flooding the users’ own blocks still
323 leaked 1662 messages under the counter and cost 0.72 points (0.97 under the
324 default). Counter and sequential tests together leaked least in total (28 to
325 82). The token bucket leaked 84 to 212, and its larger burst spared users the

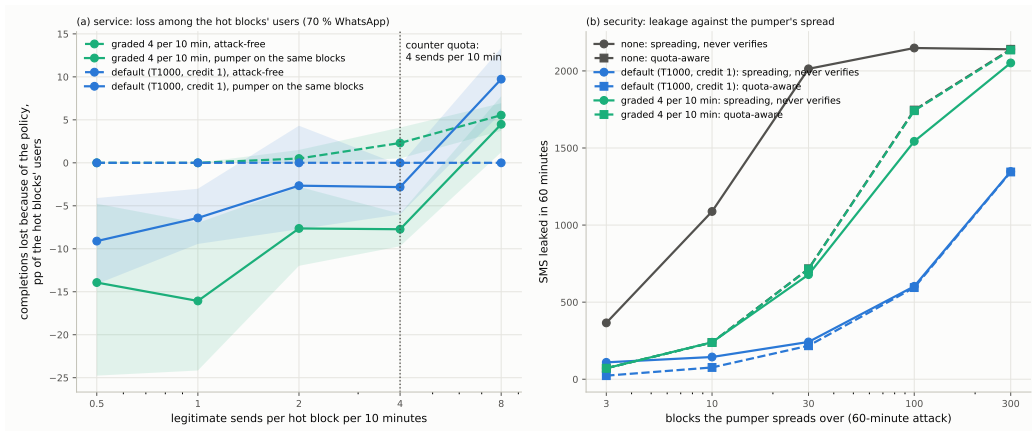


Figure 7: The counter’s operating boundary, five new seeds. (a) Attributable loss among users of three hot blocks (70% WhatsApp reachability) against their legitimate rate, without and with a never-verifying pumper on the same blocks; dotted: the quota. (b) Leakage in 60 minutes against the blocks a pumper uses, at full rate (circles) or paced to the quota (squares).

poisoner’s cost only by letting the flood through (1 974 messages, as with no policy).

Attack-free (Figure 7a), the counter costs a block’s users nothing while their own traffic stays at one send per ten minutes, 2.3 points at four and 5.5 at eight (13.2 without WhatsApp); a 30-minute product launch at three times the normal rate costs 3.7 points of all users and 16 of the launch blocks’ users. Over 60 minutes its allowance grows with the blocks a pumper uses (Figure 7b): 71 messages on 3 blocks, 679 on 30 and 2 052 on 300, against 110, 242 and 1 347 for the default, and a pumper pacing to the quota leaks what it leaks with no policy. Against the adaptive attackers the counter held the threshold-aware carrier and the receipt faker to 24 messages each (665 under the default) and the concentrated trust builder to 36 (502), but not trust builders on random numbers (577, as under the default). Over 360 minutes the human-like carrier gained 81 messages an hour under the counter, 789 under the default.

7.7. RQ5: what destination policies cost legitimate users

On one block, 20 000 Monte Carlo trials per cell give the probability of a verdict within 30 legitimate sends: 0.09% [0.06, 0.14] at 80% conversion with 20% autofill, 1.9% at 65% and 19.5% at 50%. Over 24 hours of legitimate

345 traffic the default raises 0.5 verdict events a day on 200 busy blocks at 80%
346 conversion and 5.8 at 65%, meeting 27 of about 28 800 requests; on 1 000
347 blocks, 1.6 and 17.9. Claim C4 (at most one verdict per 1000 requests at
348 conversion of 0.75 or more) held in 18 of 18 cells. A thirty-minute outage
349 on one prefix raises 0.6 to 0.8 verdict events, against 1.8 with send-clocked
350 tests.

Table 10: The block poisoner against the sequential tests, caps lifted, ten seeds; 20-minute runs except recovery (70 minutes, attack stops at minute 10). Verdict events (stage 1 / 2), legitimate requests that met a verdict, those never completed (descriptive), and completions lost to enforcement against the observe-only run on the same trace (attributable).

Variant, WhatsApp	Events	Hit	Never compl.	Attributable (points)	Compl.
graded, 70%	16.2 (12.9 / 3.3)	98.4	32.9	12.7 [4.6, 22.9]	3.09 75.9%
graded, 0%	16.7 (12.9 / 3.8)	98.4	46.0	25.8 [7.4, 49.1]	6.26 72.7%
graded, 100%	16.1 (12.9 / 3.2)	98.4	26.6	6.4 [3.1, 9.8]	1.56 77.4%
24-h denylist, 70%	17.9	121.2	121.2	97.7 [52.0, 146.8]	23.93 55.0%
recovery, whole run	11.4 (9.0 / 2.4)	420.5	136.1	48.2 [17.1, 84.3]	3.39 75.5%
after the stop	–	406.5	–	46.2 [16.9, 79.8]	– –

351 An attacker can aim verdicts at legitimate users (Table 10): flooding the
352 twenty blocks they concentrate on costs 3.1% of users, 6.3% without fallback
353 and 23.9% with the denylist. A poisoner that stops after ten minutes leaves
354 verdicts that meet 406.5 requests after the stop, an attributable loss of 46.2
355 in that period (48.2 over the whole run). Grading limits the harm; it caps it
356 neither at a challenge nor at the attack’s duration.

357 7.8. Adaptive attackers, economics and overhead

358 These scripted adaptations are not an optimised adversary; their leakages
359 are not bounds. Against v2, a datacenter attacker buying every challenge
360 solution leaks 55; the trust builder’s 500 pairs leak 263 while verifying, then
361 253 of 289 flood requests; a threshold-aware carrier and a receipt faker leak
362 570 each with no verdict. Receipt-robust tests stop the receipt faker at 18
363 but raise 30 false verdict events a day instead of 1 on blocks with a poor
364 route; a trust budget of eight exempt requests a minute trims three of four
365 trust-builder cells by 24 to 44 messages.

366 If pumpers earn a share of the 0.1422 USD retail price and pay an event-
367 level bill, the attacker breaks even at a share of 0.043 under v1; under v2, at
368 0.271 (never verifies), 0.235 (buys challenges) and 0.054 (human-like), while
369 the instant verifier would need 1.310; over 360 minutes under the counter the

370 human-like carrier would need 1.048. These rest on assumed prices and omit
371 preparation and contracts; they do not measure criminal profit.

372 On a real Redis (four vCPUs, four workers), a send costs 13.8ms at
373 the median and 21.1ms at the 99th percentile. With the 400ms floor and
374 50ms vendor calls, throughput is 67, 118 and 133 requests per second at
375 concurrency 32, 64 and 128, with 0.1, 0.7 and 30% of requests over the floor.
376 With fast vendors no pair of five outcome classes was distinguished (smallest
377 Kolmogorov–Smirnov $p = 0.05$; best single-threshold accuracy 53%). Under
378 an adversarial mixture with heavy-tailed vendors at concurrency 128, 81% of
379 sends overran the floor, and an observer seeing only its own response times
380 told a send from a refusal with 88% balanced accuracy (threshold fitted to
381 22 refusals).

382 8. Discussion

383 *Lessons.* Outcome-based destination tests were defeated by carriers that ver-
384 ify with human-like delay, fake receipts or know the thresholds. A short-
385 window send counter was not, but it lost to wide, paced and trust-building
386 pumpers and cost users whose own traffic on a block neared its quota or was
387 shared with an attacker. For an operator, the results point to a quota set
388 from each block’s own traffic, the counter and tests together, which leaked
389 least, and watching for spread, which neither contains.

390 *Limitations.* Everything is simulated and the calibration is assumed. Both
391 protocols were written by the author for the author’s generator, the second
392 knowing which counters had won. Some results follow from the generator (a
393 perfect premium list, a datacenter attacker that declines challenges, a farm
394 with human scores), and selection used an unweighted sum of four scripted
395 pumpers’ leakage and a mean service target. The implementation defects
396 of Section 6 are open, vendor adapters are untested live, and the timing
397 result holds only for the observer model tested. Replaying real traffic is the
398 experiment this paper most needs.

399 *Ethics and privacy.* Destination policies challenge people for what strangers
400 sharing their network or block did, and fingerprints are kept for 90 days;
401 deployment under the GDPR or Saudi Arabia’s PDPL needs a lawful basis
402 and a route for refused users. No human data was used.

403 9. Conclusion

404 In this simulation, at the tested settings, the design contains attacks
405 that carry a signature at little cost to simulated users, and a never-verifying
406 pumper at a cost a closed-form estimate reproduces; it rations, rather than
407 separates, a residential attacker that looks like a person. Counting sends on
408 the destination beat testing outcomes against concentrated pumpers, but a
409 frozen counter failed its service claims when the attacker shares blocks with
410 real users, and loses to wide, paced and trust-building pumpers.

411 **Artifact availability and supplementary material**

412 Code, simulation, results and scripts:
413 <https://github.com/nahmedpsu/OTP-DDoS>, release 2.8.0 (a DOI will be
414 added on acceptance). The protocols are in `config/`; `results/`
415 `study_specs.json` and `results/counter_study_specs.json` give every
416 run's configuration, `results/counter_study_per_seed.csv` per-seed
417 outcomes, and `results/headline_numbers.md` each headline number's
418 source. The full tables in `results/` and the first robustness run are the
419 supplementary material.

420 **CRedit authorship contribution statement**

421 **Naveed Ahmed:** Conceptualization, Methodology, Software, Valida-
422 tion, Investigation, Writing – original draft, Writing – review & editing.

423 **Declaration of competing interest**

424 The author declares no known competing financial interests or personal
425 relationships that could have appeared to influence this work.

426 **Declaration of generative AI and AI-assisted technologies in the** 427 **writing process**

428 The author used Claude (Anthropic) to assist with code, the evaluation,
429 and drafting and editing, checked the numbers against the recorded results
430 and the claims against the code, and takes full responsibility for the content.

431 Data availability

432 No real traffic data was used; all synthetic data, seeds and results are in
433 the repository.

434 References

- 435 [1] MITRE Corporation, Resource hijacking: SMS pumping, technique
436 T1496.003, MITRE ATT&CK, <https://attack.mitre.org/techniques/T1496/003/>, accessed 1 October 2026.
- 438 [2] Commsrisk, Elon Musk says Twitter lost \$60mn a year because 390
439 telcos used bot accounts to pump A2P SMS, <https://commsrisk.com/elon-musk-says-twitter-lost-60mn-a-year-because-390-telcos-used-bot-accounts-to-pump-a2p-sms/>, trade press; the figure
441 is the company’s own claim (2023).
- 443 [3] J. H. Huh, H. Shin, S. Ahn, H. Yi, J. Cho, T. Kim, M. Lim, N. Choi,
444 Preventing artificially inflated SMS attacks through large-scale traffic in-
445 spection, in: 34th USENIX Security Symposium, USENIX Association,
446 Seattle, WA, 2025.
- 447 [4] M. Sahin, A. Francillon, P. Gupta, M. Ahamad, SoK: Fraud in telephony
448 networks, in: 2017 IEEE European Symposium on Security and Privacy
449 (EuroS&P), IEEE, 2017. doi:10.1109/EuroSP.2017.40.
- 450 [5] M. Sahin, A. Francillon, Understanding and detecting international rev-
451 enue share fraud, in: Network and Distributed System Security Sym-
452 posium (NDSS), Internet Society, 2021. doi:10.14722/ndss.2021.24051.
- 453 [6] Twilio Inc., Verify fraud guard, [https://www.twilio.com/docs/veri-
454 fy/preventing-toll-fraud/sms-fraud-guard](https://www.twilio.com/docs/verify/preventing-toll-fraud/sms-fraud-guard), product documenta-
455 tion. Accessed 1 October 2026.
- 456 [7] Twilio Inc., SMS pumping risk score (Lookup v2), [https://www.twil-
457 io.com/docs/lookup/v2-api/sms-pumping-risk](https://www.twilio.com/docs/lookup/v2-api/sms-pumping-risk), product documen-
458 tation. Accessed 1 October 2026.
- 459 [8] J. Heinanen, R. Guerin, A single rate three color marker, RFC 2697,
460 Internet Engineering Task Force (1999). doi:10.17487/RFC2697.

- 461 [9] M. Motoyama, K. Levchenko, C. Kanich, D. McCoy, G. M. Voelker,
462 S. Savage, Re: CAPTCHAs – understanding CAPTCHA-solving ser-
463 vices in an economic context, in: 19th USENIX Security Symposium,
464 USENIX Association, 2010.
- 465 [10] A. Dionysiou, E. Athanasopoulos, SoK: Machine vs. machine – A sys-
466 tematic classification of automated machine learning-based CAPTCHA
467 solvers, *Computers & Security* 97 (2020) 101947. doi:10.1016/j.cose
468 .2020.101947.
- 469 [11] D. Dean, A. Stubblefield, Using client puzzles to protect TLS, in: 10th
470 USENIX Security Symposium, USENIX Association, 2001.
- 471 [12] X. Mi, X. Feng, X. Liao, B. Liu, X. Wang, F. Qian, Z. Li, S. Alrizah,
472 L. Chen, Y. Qian, Resident evil: Understanding residential IP proxy
473 as a dark service, in: 2019 IEEE Symposium on Security and Privacy
474 (S&P), IEEE, 2019. doi:10.1109/SP.2019.00011.
- 475 [13] P. Richter, F. Wolsing, A. Chandrasekaran, R. Padmanabhan, M. All-
476 man, R. Bush, A. Feldmann, A multi-perspective analysis of carrier-
477 grade NAT deployment, in: *Proceedings of the 2016 ACM Internet
478 Measurement Conference (IMC)*, ACM, 2016. doi:10.1145/298744
479 3.2987474.
- 480 [14] J. R. Douceur, The Sybil attack, in: *Peer-to-Peer Systems (IPTPS
481 2002)*, Vol. 2429 of LNCS, Springer, 2002, pp. 251–260. doi:10.100
482 7/3-540-45748-8_24.
- 483 [15] A. Wald, Sequential tests of statistical hypotheses, *The Annals of Math-
484 ematical Statistics* 16 (2) (1945) 117–186. doi:10.1214/aoms/1177731
485 118.
- 486 [16] E. S. Page, Continuous inspection schemes, *Biometrika* 41 (1/2) (1954)
487 100–115. doi:10.2307/2333009.
- 488 [17] V. V. Veeravalli, T. Banerjee, Quickest change detection,
489 arXiv:1210.5552 (book chapter) (2012).
- 490 [18] NIST/SEMATECH, CUSUM control charts, section 6.3.2.3, e-handbook
491 of statistical methods, [https://www.itl.nist.gov/div898/handbook](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm)
492 [/pmc/section3/pmc323.htm](https://www.itl.nist.gov/div898/handbook/pmc/section3/pmc323.htm), accessed 2 October 2026.

- 493 [19] S. R. Howard, A. Ramdas, J. McAuliffe, J. Sekhon, Time-uniform, non-
494 parametric, nonasymptotic confidence sequences, *The Annals of Statis-*
495 *tics* 49 (2) (2021) 1055–1080. doi:10.1214/20-AOS1991.
- 496 [20] S. Wiefling, L. Lo Iacono, M. Dürmuth, Is this really you? An empirical
497 study on risk-based authentication applied in the wild, in: *ICT Systems*
498 *Security and Privacy Protection (IFIP SEC 2019)*, Vol. 562 of *IFIP Ad-*
499 *vances in Information and Communication Technology*, Springer, 2019,
500 pp. 134–148. doi:10.1007/978-3-030-22312-0_10.
- 501 [21] C. Mulliner, R. Borgaonkar, P. Stewin, J.-P. Seifert, SMS-based one-
502 time passwords: Attacks and defense, in: *Detection of Intrusions and*
503 *Malware, and Vulnerability Assessment (DIMVA 2013)*, Vol. 7967 of
504 *LNCS*, Springer, 2013, pp. 150–159. doi:10.1007/978-3-642-39235-
505 -1_9.
- 506 [22] H. Siadati, T. Nguyen, P. Gupta, M. Jakobsson, N. Memon, Mind your
507 SMSes: Mitigating social engineering in second factor authentication,
508 *Computers & Security* 65 (2017) 14–28. doi:10.1016/j.cose.2016.
509 09.009.
- 510 [23] N. Ahmed, OTP Guard: a layered SMS validation, risk-scoring and
511 rate-limiting pipeline against OTP flood abuse (version 2.8.0), <https://github.com/nahmedpsu/OTP-DDOS>, source code, evaluation and
512 recorded results. Archived DOI to be added (2026).
513
- 514 [24] Google LLC, SMS retriever API overview, <https://developers.google.com/identity/sms-retriever/overview>, developer documenta-
515 tion. Accessed 1 October 2026.
516
- 517 [25] Web Incubator Community Group, WebOTP API, <https://github.com/WICG/WebOTP>, specification repository. Accessed 1 October 2026.
518
- 519 [26] Twilio Inc., Verify API, <https://www.twilio.com/en-us/user-authentication-identity/verify>, product page. Accessed 3 October 2026.
520